Chapter 13

Security primitives

pick the right tool; never invent your own

Security code fails silently. A broken sort gives wrong output you'll notice; a broken password check works perfectly until someone steals the database. So this chapter is less about clever code and more about **choosing the right primitive** and using it exactly as intended.

Start every security task with one question: **what am I protecting against?**

I need to...	use	module
detect accidental change (a corrupted download)	a hash	`hashlib`
prove a message came from me and wasn't edited	a MAC (keyed hash)	`hmac`
store passwords so a stolen database doesn't leak them	a slow, salted KDF	`hashlib.scrypt`
make an unguessable token, reset link or key	a CSPRNG	`secrets`
keep data secret (only key holders can read it)	authenticated encryption	**not in stdlib**: `cryptography`
talk to a server without eavesdroppers	TLS	`ssl` (via `urllib`, `http.client`)
put bytes in text (JSON, URLs, email)	an **encoding**, not security	`base64`
name things uniquely	an id, **not a secret**	`uuid`

RULE OF THUMB

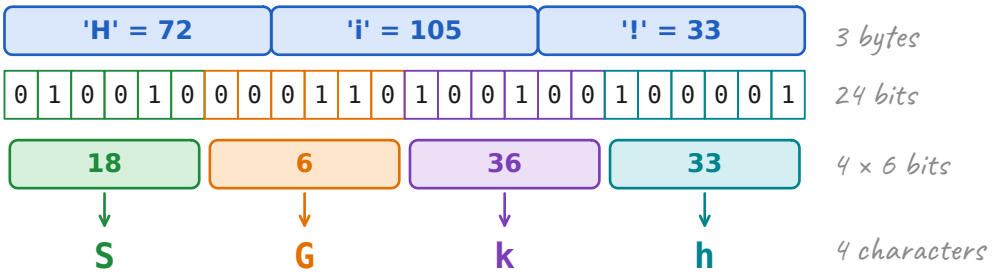
Never design your own crypto ("I'll XOR it with the key", "I'll hash it twice"). Even experts get it wrong. Combine well-known primitives in well-known ways, as this chapter does.

Everything here speaks bytes

Every primitive in this chapter takes and returns `bytes`, not `str`. Text becomes bytes with `.encode()` (UTF-8 by default); digests and keys come out as raw bytes, which you usually show as **hex** or **base64**:

```
>>> import hashlib, base64
>>> raw = hashlib.sha256("hello".encode()).digest()
>>> len(raw), raw[:6]
(32, b',\xf2M\xba_\xb0')
>>> raw.hex()[:24]
'2cf24dba5fb0a30e26e83b2a'
>>> base64.b64encode(raw)[:16]
b'LPJNul+wow4m6Dsq'
```

Base64 turns any bytes into 64 safe characters (A–Z a–z 0–9 + /), so binary data can travel in JSON, URLs or email. Every 3 bytes become 4 characters:




```
>>> base64.b64encode(b"Hi!"), base64.b64encode(b"Hi")
(b'SGkh', b'SGk=')
>>> base64.urlsafe_b64encode(b"\xfb\xff"), base64.b64encode(b"\xfb\xff")
(b'-_8=', b'+/8=')
>>> base64.b64decode("SGkh")
b'Hi!'
```

TRAP

Base64 is **not encryption**. Anyone can decode it in one line, as above. If you see a "secret" stored as base64, it's stored in plain text. Use `urlsafe_b64encode` (- and _ instead of + and /) for anything that goes in a URL; `=` is padding to a multiple of 4.

hashlib: a fingerprint for data

A cryptographic hash turns any amount of data into a fixed-size fingerprint. Four properties make it useful:

property	means	so you can
deterministic	same input → same digest, always, everywhere	compare files without sending them
fixed size	1 byte or 10 GB → 32 bytes (SHA-256)	store or index fingerprints cheaply
one-way	can't get the input back from the digest	publish a digest safely
avalanche	change 1 bit → about half the output bits flip	spot any edit at all

```
1 import hashlib
2
3 a = hashlib.sha256(b"transfer 100 to alice").hexdigest()
4 b = hashlib.sha256(b"transfer 900 to alice").hexdigest()
5 print(a[:32])
6 print(b[:32])
7 flipped = bin(int(a, 16) ^ int(b, 16)).count("1")
8 print(f"{flipped} of 256 bits differ")
```

← one character changed

← count differing bits

```
766e90601b2e01f7c7a633d6368a07a5
eb9535686c306af9c27094fd36a17273
116 of 256 bits differ
```

For big files, feed the hash in chunks with `.update()`, so memory use stays tiny however large the file is:

```
1 with open("big.bin", "wb") as out:
2     out.write(b"x" * 5_000_000)
3
4 def file_sha256(path, chunk=64 * 1024):
5     h = hashlib.sha256()
6     with open(path, "rb") as fh:
7         while block := fh.read(chunk):
8             h.update(block)
9     return h.hexdigest()
10
11 print(file_sha256("big.bin")[:16], len(hashlib.sha256().digest()), "bytes")
```

```
03a7bd518f3e4eca 32 bytes
```

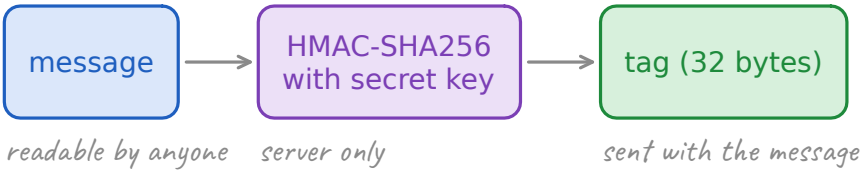
line 7: 64 KB at a time

line 8: same result as one big call

algorithm	status
`sha256`, `sha512`, `sha3_256`, `blake2b`	good for integrity and fingerprints
`md5`, `sha1`	**broken** for security (collisions can be manufactured); still fine as a non-security checksum
any of these for passwords	**wrong tool**: they're designed to be fast (next pages)

hmac: a hash only the key holder can make

A plain hash proves nothing about **who** made it: an attacker who edits a message just recomputes the hash. An **HMAC** mixes a secret key into the hash, so only someone with the key can produce a valid tag.




```python
import hmac, hashlib

KEY = b"k3y-from-an-env-variable"
msg = b"user=42;role=student"
tag = hmac.new(KEY, msg, hashlib.sha256).hexdigest()

def valid(msg, tag):
    expected = hmac.new(KEY, msg, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, tag)

print(valid(msg, tag))
print(valid(b"user=42;role=admin", tag))
print(valid(msg, hashlib.sha256(msg).hexdigest()))
```

← never ==

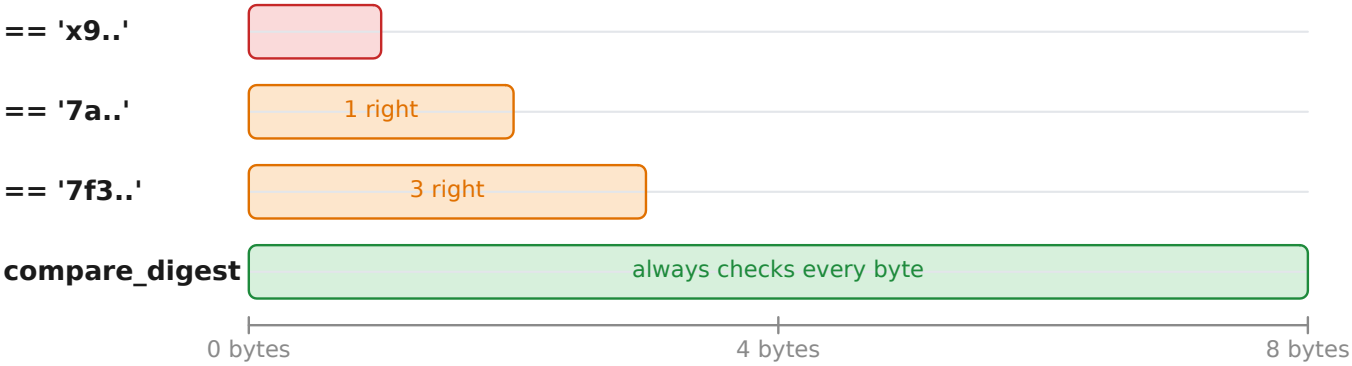
← edited: False

← no key: can't forge

```
True
False
False
```

Why compare_digest and not ==

`==` on strings stops at the **first differing character**. An attacker who can time thousands of requests sees that a guess with the right first character takes slightly longer, then works out the tag one character at a time. `compare_digest` always takes the same time. Say the real tag starts `7f3`:

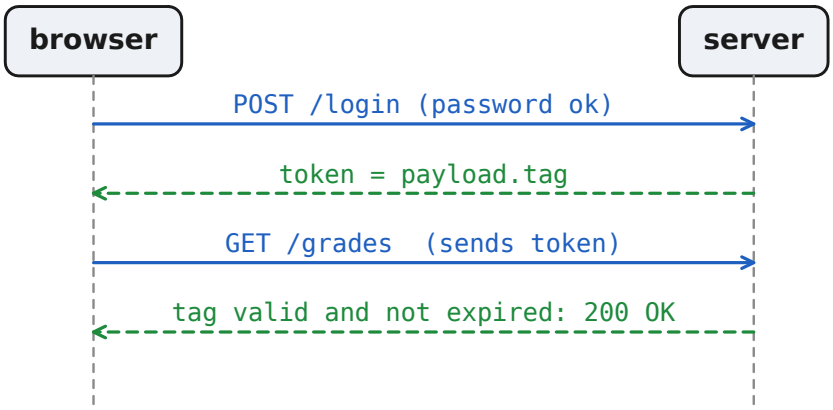


KEY IDEA

A MAC gives **integrity + authenticity**, not secrecy. The message is still readable; it just can't be changed without detection.

Building a signed token

Put the last two pages together and you have the idea behind login cookies, password-reset links and JWTs: **data + expiry, base64url-encoded, with an HMAC tag**. The server gives it out, the client hands it back, and the server can trust it without a database lookup.



```python
import base64, hashlib, hmac, json, time

KEY = b"server-secret"

def b64(b: bytes) -> str:
    return base64.urlsafe_b64encode(b).rstrip(b"=").decode()

def unb64(s: str) -> bytes:
    return base64.urlsafe_b64decode(s + "=" * (-len(s) % 4))

def issue(user, ttl, now):
    payload = b64(json.dumps({"u": user, "exp": now + ttl}).encode())
    tag = b64(hmac.digest(KEY, payload.encode(), "sha256"))
    return f"{payload}.{tag}"

def check(token, now):
    payload, _, tag = token.partition(".")
    good = b64(hmac.digest(KEY, payload.encode(), "sha256"))
    if not hmac.compare_digest(good, tag):
        raise ValueError("bad signature")
    data = json.loads(unb64(payload))
    if now >= data["exp"]:
        raise ValueError("expired")
    return data["u"]

t = issue("ana", ttl=3600, now=1_000_000)
print(t)
print(check(t, now=1_000_100))
forged = b64(b'{"u": "bob", "exp": 9999999}') + "." + t.split(".")[1]
for bad, when in [(forged, 1_000_100), (t, 1_004_000)]:
    try:
        check(bad, when)
    except ValueError as e:
        print("refused:", e)
print(unb64(t.split(".")[0]))
```

```
eyJ1IjogImFuYSIsICJleHAiOiAxMDAzNjAwfQ.ZRGnPrzceX2Gc9cMguMbjPTH2q-nlOKI-QIqckDMQx8
ana
refused: bad signature
refused: expired
b'{"u": "ana", "exp": 1003600}'
```

TRAP

The last line shows the payload is **readable by anyone**: signed is not encrypted. Never put secrets in a token's payload. And always check the signature **before** trusting anything inside, including the expiry.

secrets: randomness an attacker can't predict

The `random` module is a Mersenne Twister: fast, great for simulations and games, and completely **predictable**. Anyone who sees 624 outputs can compute every future one. Even easier, a seed reproduces the whole stream:

random: reproducible (bad for secrets)

```
import random
random.seed(2026)
print(random.randint(0, 10**6))
random.seed(2026)
print(random.randint(0, 10**6))
```

```
124906
124906
```

secrets: from the OS (os.urandom)

```
import secrets
a = secrets.randbelow(10**6)
b = secrets.randbelow(10**6)
print(a != b or "astronomically rare")
print(len(secrets.token_bytes(16)))
```

```
True
16
```

need	call	gives
a URL-safe token (reset link, session id)	`secrets.token_urlsafe(32)`	43 chars, 256 bits
a hex token (API key)	`secrets.token_hex(16)`	32 hex chars, 128 bits
raw key bytes	`secrets.token_bytes(32)`	32 bytes
pick from a list (password chars)	`secrets.choice(seq)`	one item
a number in [0, n)	`secrets.randbelow(n)`	an int

```
1  import secrets, string
2
3  def make_password(n=16):
4      alphabet = string.ascii_letters + string.digits + "!@#$%&*-_"
5      while True:
6          pw = "".join(secrets.choice(alphabet) for _ in range(n))
7          if (any(c.islower() for c in pw) and any(c.isupper() for c in pw)
8                  and any(c.isdigit() for c in pw)):
9              return pw
10
11 print(len(make_password()), len(secrets.token_urlsafe(32)))
```

```
16 43
```

line 6: secrets, not random

line 7: retry until every class appears

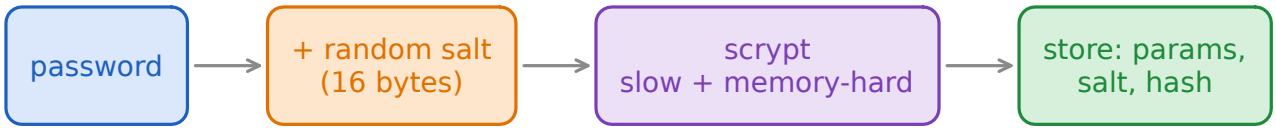
RULE OF THUMB

If a person or attacker must not guess it, use `secrets`. Tokens, keys, salts, one-time codes, temporary passwords. `random` is for everything else.

Storing passwords: salted and deliberately slow

Assume the database **will** be stolen one day. Then the question is how long it takes the attacker to recover passwords from what you stored. Each step below closes one attack:

you store	attacker does	verdict
the password	reads it	**never**
`sha256(password)`	looks it up in a precomputed table, or tries billions per second on a GPU	**still broken**
`sha256(salt + password)`	can't use tables (each user has a random salt), but still billions of guesses per second	**better, not enough**
`scrypt(password, salt, n, r, p)`	each guess costs ~40 ms and 16 MB of memory	**right**




```python
import hashlib, hmac, secrets, time

N, R_, P = 2**14, 8, 1

def hash_password(pw: str) -> str:
    salt = secrets.token_bytes(16)
    h = hashlib.scrypt(pw.encode(), salt=salt, n=N, r=R_, p=P)
    return f"scrypt${N}${R_}${P}${salt.hex()}${h.hex()}"

def verify_password(pw: str, stored: str) -> bool:
    _, n, r, p, salt, h = stored.split("$")
    got = hashlib.scrypt(pw.encode(), salt=bytes.fromhex(salt),
                         n=int(n), r=int(r), p=int(p))
    return hmac.compare_digest(got.hex(), h)

t = time.perf_counter()
s = hash_password("correct horse battery staple")
ms = (time.perf_counter() - t) * 1000
print(s[:44] + "...")
print(verify_password("correct horse battery staple", s),
      verify_password("Tr0ub4dor&3", s))
print(hash_password("same") != hash_password("same"), f"{ms:.0f} ms per hash")
```

```
scrypt$16384$8$1$22338bfc68b892b3bf0f5085d76...
True False
True 35 ms per hash
```

The parameters are stored **with** each hash, so you can raise the cost next year and old hashes still verify (re-hash them when the user next logs in). `hashlib.pbkdf2_hmac('sha256', pw, salt, 600_000)` is the other stdlib option; `argon2` (a pip package) is the current favourite.

TRAP

The slowness is the point. 40 ms is nothing for one login and ruinous for an attacker trying a billion guesses. Don't "optimise" it away, and do rate-limit login attempts too.

Encryption: the one thing the stdlib doesn't do

To keep data **secret** (only key holders can read it) you need encryption, and Python's standard library deliberately ships none: no AES. The standard answer is the `cryptography` package and its **Fernet** recipe, which bundles AES encryption with an HMAC so tampering is detected too ("authenticated encryption"). It's the first pip package in this course; Chapter 19 covers choosing and installing packages safely.

```
$ python3 -m venv .venv && source .venv/bin/activate
$ pip install cryptography
```

```python
from cryptography.fernet import Fernet, InvalidToken

key = Fernet.generate_key()          # 32 random bytes, base64: keep it SECRET
box = Fernet(key)

token = box.encrypt(b"exam room B12, 09:00")
print(token[:28], b"...")
print(box.decrypt(token))

tampered = token[:-2] + (b"A" if token[-2:-1] != b"A" else b"B") + token[-1:]
try:
    box.decrypt(tampered)
except InvalidToken:
    print("tampered: refused")
try:
    Fernet(Fernet.generate_key()).decrypt(token)
except InvalidToken:
    print("wrong key: refused")
```

```
b'gAAAAABqxDQyATNi38yoKziUdQA0' b'...'
b'exam room B12, 09:00'
tampered: refused
wrong key: refused
```

key management question	sensible default
where does the key live?	an environment variable or secrets file, **never in the source or git**
what if it leaks?	generate a new one and re-encrypt (`MultiFernet` helps rotate)
what if it's lost?	the data is gone. Back keys up separately from the data.

HONEST NOTE

Most programs don't need encryption code at all. Data in transit is TLS's job (next page); data at rest is often better handled by disk encryption. Reach for Fernet when one specific field must stay secret even from someone holding the database file.

ssl: TLS, the lock icon

TLS (what `https://` uses) encrypts the connection **and** proves you're talking to the real server, by checking its certificate against trusted authorities. `ssl.create_default_context()` turns all the checks on; `urllib.request` and `http.client` use it automatically:

```
import ssl
ctx = ssl.create_default_context()
print(ctx.verify_mode == ssl.CERT_REQUIRED, ctx.check_hostname)
print(ctx.minimum_version.name, ssl.OPENSSL_VERSION)
```

```
True True
TLSv1_2 OpenSSL 3.5.5 27 Jan 2026
```

TRAP

Tutorials "fix" certificate errors with `ssl._create_unverified_context()`, `CERT_NONE` or `verify=False`. That keeps the encryption but **drops the identity check**, so anyone in the middle (café Wi-Fi) can impersonate the server. Fix the real cause (wrong clock, missing CA bundle, wrong hostname) instead.

uuid: unique, but not secret

```
>>> import uuid
>>> uuid.uuid4().version, len(str(uuid.uuid4()))
(4, 36)
>>> a, b = uuid.uuid7(), uuid.uuid7()
>>> a < b
True
```

`uuid4` is random, `uuid7` (new in 3.14) starts with a timestamp, so ids sort by creation time, which databases like. Use them as **names**. For anything that grants access, use `secrets`.

Checklist, card, practice

- Bytes in, bytes out: `.encode()` text going in; `.hex()` or base64 to display.
- Integrity only: `hashlib.sha256`. Integrity + who: `hmac` with a secret key.
- Compare secrets with `hmac.compare_digest`, never `==`.
- Tokens, salts, keys: `secrets`. Never `random`.
- Passwords: `hashlib.scrypt` with a per-user salt; store the parameters.
- Secrecy: `cryptography`'s Fernet; the key lives outside the code.
- Never disable certificate checks. Signed ≠ encrypted. Base64 ≠ encrypted.

LIBRARY CARD **`hashlib · hmac · secrets`**

Job	Fingerprints (`hashlib`), keyed tags (`hmac`), unguessable randomness (`secrets`).
Core objects	`hashlib.sha256(b)` → hash object: `.update(b)`, `.digest()`, `.hexdigest()`. `hmac.new(key, msg, 'sha256')`, `hmac.digest(...)`, `hmac.compare_digest(a, b)`. `secrets.token_urlsafe(n)`, `choice`, `randbelow`.
Lifecycle	new → update… → digest. No closing. Keys come from outside the code.
Errors	`TypeError` for `str` instead of `bytes`; `ValueError` for unknown algorithms / bad scrypt parameters.
Traps	== on tags (timing) · md5/sha1 · fast hashes for passwords · `random` for tokens · thinking base64 or signing hides data.
Docs	docs.python.org/3/library/{hashlib,hmac,secrets}.html · OWASP Password Storage Cheat Sheet

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 13.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 13.1` shows other valid solutions and their trade-offs.

13.1 build from a spec Fingerprint a file in chunks
13.2 build from a spec Sign and verify messages with HMAC
13.3 build from a spec Store passwords properly
13.4 build from a spec A password generator you'd trust
13.5 learn an unseen module URL-safe base64 without padding
13.6 mini-project Mini-project: expiring signed tokens

PASSPORT STAMP ask the same question of every new language (Appendix B)

12. Stdlib and tooling

Security primitives ship in the stdlib (`hashlib`, `hmac`, `secrets`, `ssl`) except symmetric encryption, which comes from the `cryptography` package.

6. Data structures

`bytes` vs `str` is a hard boundary: crypto, sockets and files in binary mode all speak bytes.

DEEP DIVE (your vault)

Vault notes: 19.16 Cryptographic objectives and the rule against invention · 19.17 Hash functions · 19.18 Message authentication codes · 19.19 Symmetric authenticated encryption · 19.21 Password storage · 19.22 Cryptographic randomness · 19.23 Key management · 5.12 Bytes and the text boundary.